Duolingo Web App

SDE Fullstack Assignment

Description

Build a functional clone of the Duolingo web application that replicates Duolingo's design, user experience, and core lesson and gamification workflows.

The platform should let a learner move through a skill tree / learning path, complete lessons made of varied interactive exercises, earn XP and maintain a streak, lose and regain hearts, and track progress, all within the playful, gamified interface of the original Duolingo app.

Your implementation should visually and functionally feel like a modern Duolingo. The focus is on recreating the lesson loop and gamification mechanics; actual language content can be small and seeded.

Note: Your application should match the original application in terms of UI and UX, look and feel should be exactly the same.

AI Tools Usage

You are allowed and encouraged to use AI tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI assistant for development. Use AI as heavily as you like to move fast. However, you must understand every line of code you submit and be prepared to explain your implementation decisions during the evaluation interview.

Technical Stack

- Frontend: Next.js (TypeScript)
- Backend: Python with FastAPI / Django
- Database: SQLite (design your own schema)

Note: you only need a small amount of seeded course content (one language, a handful of skills). Audio can be optional/placeholder.

Core Features (Must Have)

1. Learning Path / Skill Tree

Recreate the Duolingo home path.

- A visual path/tree of units and skills with lock/unlock progression
- Completed vs available vs locked states
- Progress rings/crowns per skill
- Top bar showing streak, XP, hearts, and (mocked) gems

2. Lesson Player (the core loop)

Implement a lesson made of a sequence of exercises.

- Multiple exercise types: multiple choice, translate (word bank/tap-the-words), match pairs, fill in the blank, and type-the-answer
- Immediate correct/incorrect feedback with the signature feedback bar
- Progress bar across the lesson
- Hearts: lose one on a wrong answer; lesson end/failure handled
- Award XP and mark the skill's progress on completion

3. Gamification & Progress

- Streak counter that increments on daily activity (day logic can be simulated/testable)
- XP totals and a simple leaderboard (can be seeded)
- Hearts regeneration over time or via a mocked “practice/refill”
- Daily goal / XP goal indicator
- All progress (XP, streak, hearts, completed skills) must persist per user

4. Content Management

- Course content (units, skills, lessons, exercises) stored in the database and seeded
- A learner profile page with stats (streak, total XP, achievements)
- All learner progress must persist

5. Duolingo Experience

The application should closely resemble the Duolingo experience, including:

- Playful, colorful, gamified UI with mascot-style flourishes
- The lesson player with animated feedback
- Modals (lesson complete, out of hearts), toasts, and celebratory states
- Path navigation and progress visuals
- Settings placeholders

The goal is to make the application feel like Duolingo rather than a generic quiz app.

Mocked / Placeholder Sections

The following can be present as placeholders (a simple “Coming Soon” is sufficient):

- Real speech recognition / pronunciation exercises
- In-app purchases / Super subscription (gems can be mocked)
- Friends / social features (leaderboard can be seeded)
- Multiple languages (one seeded language is enough)
- Real user authentication may be simplified (assume a default logged-in learner)

Bonus (Optional)

- Audio for exercises (text-to-speech or seeded audio)
- Achievements / badges system
- Real functioning leaderboard across seeded users
- Timed practice / “legendary” challenge mode
- Dark mode

- Responsive design (mobile, tablet, desktop)

Important Notes

- UI Design: your application should totally resemble Duolingo's design. Study Duolingo's UI carefully before starting.
- Sample Data: seed your database. Seed one language course with a few units/skills/lessons and varied exercises, plus a sample learner with some progress, so the app is immediately usable.
- Database Design: design your own database schema. This will be evaluated.
- README File: include setup instructions, tech stack used, architecture overview, database schema, and any assumptions made.
- Original Work: plagiarism from existing repositories will result in immediate disqualification.

Deliverables

- Source Code: a public GitHub repository containing frontend/ and backend/.
- Documentation: a README with setup instructions, architecture overview, database schema, and API overview.
- Demo: a hosted, working link.

Submission

- Upload your code to GitHub and ensure the repository is public.
- Deploy your application (Vercel, Netlify, Render, Railway, or any cloud service).
- Submit both the GitHub repository link and the deployed application link.

Evaluation Criteria

Criteria	What We Look For
Functionality	All core features working correctly, including the lesson loop and gamification (XP, streak, hearts)
UI/UX	Visual similarity to the original app's design and UX patterns
Database Design	Well-structured schema with proper relationships
Backend / API Design	Clean, sensible API design and architecture
Code Quality	Clean, readable, and well-organized code
Code Modularity	Proper separation of concerns, reusable components
Code Understanding	Ability to explain your code during evaluation

Timeline

Estimated effort: approximately 24 hours of work.
Submission Deadline: as communicated alongside this assignment.
Good luck!